



MXEN4000
Mechatronic Engineering Research Project 1

Gauss-Newton Matrix

Github Repo:
<https://github.com/soapydoor17/satellite-orbit-deteremination>

Tobey Dore
21480175

October 1, 2026

Contents

1	Objective	1
2	Theory	1
3	Method	4
3.1	Inputs	4
4	Results	5
4.1	Single Doppler Measurement	5
4.2	Multiple Doppler Measurements	6
4.3	Standardisation	7
4.3.1	Single Doppler Measurement	8
4.3.2	Multiple Doppler Measurements	8
5	Discussion	9
5.1	Implications for Project	10
A	Python Code	11
B	Fake Doppler Measurement Data	18
	References	19

List of Figures

1	Block diagram of main code	4
---	--------------------------------------	---

Listings

tobeyLibrary.py	11
---------------------------	----

1 Objective

Using the gradient descent equations for correcting initial mean anomaly M_0 and semi-major axis a from Doppler measurements, create the Gauss-Newton Matrix. For the typical quadratic loss gradient descent problem, this matrix is approximately equivalent to the 'Fisher Information Matrix', which shows how observable each parameter is from the data. This is usually done through get the eigenvectors/values of the matrix. Compare the observability of the parameters of a single data point vs multiple data points.

2 Theory

The Gauss-Newton Matrix is given by:

$$G = \frac{1}{N} J^T J$$

Where N is the number of data points being considered and J is the Jacobian of the Doppler residual, $\epsilon = \Delta f_D = f_{D \text{ pred}} - f_{D \text{ true}}$, with respect to the parameters, $\theta = [M_0, a]$. The Jacobian is an $N \times 2$ matrix as such:

$$J(\theta) = \frac{\partial \epsilon}{\partial \theta} = \begin{bmatrix} \frac{\partial \epsilon_1}{\partial M_0} & \frac{\partial \epsilon_1}{\partial a} \\ \frac{\partial \epsilon_2}{\partial M_0} & \frac{\partial \epsilon_2}{\partial a} \\ \vdots & \vdots \\ \frac{\partial \epsilon_n}{\partial M_0} & \frac{\partial \epsilon_n}{\partial a} \end{bmatrix}$$

In order to calculate these partial derivatives, first the state vector of the satellite, $\mathbf{X} = [\mathbf{r}, \mathbf{v}]$, must first be found from the orbital parameters COE = $[a, e, i, \Omega, \omega, \nu]$. This is done using the COE2RV algorithm (Vallado 2022, 116). In this algorithm, the semi-parameter (p) is used instead of the semi-major axis (a). Further, the true anomaly ν should be calculated beforehand, using the Newton-Raphson Method to find the eccentric anomaly E (Vallado 2022, 65):

$$p = a(1 - e^2)$$

$$\nu = \arctan \left(\frac{\sin(E) \sqrt{1 - e^2}}{\cos(E) - e} \right)$$

Next, the ECI of the ground station at each timestamp must be calculated. Kelso (1996) shows how to convert from geodetic coordinates (latitude/longitude/altitude) to ECI:

$$\mathbf{r}_{gs} = \begin{bmatrix} R_{\oplus} C \cos \varphi \cos \theta \\ R_{\oplus} C \cos \varphi \sin \theta \\ R_{\oplus} S \sin \varphi \end{bmatrix} \quad \mathbf{v}_{gs} = \omega_{\oplus} \times \mathbf{r}_{gs}$$

Where:

- $R_{\oplus}$ is Earth's equatorial radius (6,378.137 km as given by Vallado (2022))
- $\omega_{\oplus}$ is Earth's angular velocity (7.2921159×10^{-5} rad/s as given by Vallado (2022))
- φ is the geodetic latitude
- θ is the local sidereal time
- C and S are the oblateness correction factors where:

$$- C = \frac{1}{\sqrt{1 - f(2 - f) \sin^2 \varphi}}, \quad S = (1 - f)^2 C$$

$$- f = \frac{1}{298.257} \text{ is the Earth flattening as defined by Vallado (2022)}$$

Luckily, python libraries can be used to complete this calculation instead, meaning all we need to know is the geodetic coordinates of the ground station, and the current time (see Appendix A for code on formatting timestamps):

```

1 from astropy.time import Time
2 from astropy.coordinates import EarthLocation, GCRS
3 import astropy.units as u
4
5 def groundstationECI(lat_deg, lon_deg, alt_m, t):
6     # Inputs:
7     #   lat_deg - latitude of groundstation in degrees
8     #   lon_deg - longitude of groundstation in degrees
9     #   alt_m   - altitude of groundstation in metres
10    #   t       - astropy.time.Time object (single time, or
11                #   array of times)
12    # Outputs:
13    #   r_gs - cartesian position vector(s) of groundstation (m)
14    #   v_gs - cartesian velocity vector(s) of groundstation (m/s)
15    station = EarthLocation(lat=lat_deg*u.deg, lon=lon_deg*u.
16                           deg, height=alt_m*u.m)
17
18    pos, vel = station.get_gcrs_posvel(obstime=t)
19
20    r_gs = pos.xyz.to(u.m).value
21    v_gs = vel.xyz.to(u.m/u.s).value
22    return r_gs, v_gs

```

From here, $\boldsymbol{\rho}$ and $\mathbf{v}_{rel}$ can be computed, and the predicted Doppler shift can be found:

$$\begin{aligned}\boldsymbol{\rho} &= \mathbf{r} - \mathbf{r}_{gs} \\ \mathbf{v}_{rel} &= \mathbf{v} - \mathbf{v}_{gs} \\ f_D &= k \hat{\boldsymbol{\rho}} \cdot (-\mathbf{v}_{rel}), \quad k = \frac{f_c}{c}\end{aligned}$$

Thus, the residual is:

$$\epsilon = \Delta f_D = f_{D \text{ pred}} - f_{D \text{ true}}$$

The relationship between the Doppler shift and satellite's cartesian state can be defined as:

$$\frac{\partial f_D}{\partial \mathbf{X}} = \left[-k \mathbf{v}_{rel}^T \left(\frac{I}{\|\boldsymbol{\rho}\|} - \frac{\boldsymbol{\rho} \boldsymbol{\rho}^T}{\|\boldsymbol{\rho}\|^3} \right), -k \hat{\boldsymbol{\rho}}^T \right]$$

And the change of the cartesian state with respect to time is:

$$\frac{d\mathbf{X}}{dt} = \left[\mathbf{v}, -\frac{\mu \mathbf{r}}{\|\mathbf{r}\|^3} \right]^T$$

The partial derivate of the Doppler shift with respect to the initial mean anomaly can be now finalised, where:

$$\begin{aligned}\frac{\partial f_D}{\partial M_0} &= \frac{\partial f_D}{\partial \mathbf{X}} \frac{d\mathbf{X}}{dt} \left(\frac{dM}{dt} \right)^{-1} \frac{\partial M}{\partial M_0} \\ \left(\frac{dM}{dt} \right)^{-1} &= \frac{T_p}{360} = \frac{\pi}{180n}, \quad n = \sqrt{\frac{\mu}{a^3}} \\ \frac{\partial M}{\partial M_0} &= 1\end{aligned}$$

And the partial derivative of the Doppler shift with respect to the semi-major axis is:

$$\begin{aligned}\frac{\partial f_D}{\partial a} &= \frac{\partial f_D}{\partial \mathbf{X}} \frac{d\mathbf{X}}{da} \\ \frac{d\mathbf{X}}{da} &= \frac{\partial \mathbf{X}}{\partial a} \Big|_M + \frac{\partial \mathbf{X}}{\partial M} \frac{\partial M}{\partial a} \\ \frac{\partial \mathbf{X}}{\partial a} \Big|_M &= \left[R(1 - e \cos E) \begin{bmatrix} \cos \nu \\ \sin \nu \\ 0 \end{bmatrix}, -\frac{n}{2 - 2e \cos E} R \begin{bmatrix} -\sin E \\ \sqrt{1 - e^2} \cos E \\ 0 \end{bmatrix} \right]^T \\ \frac{\partial \mathbf{X}}{\partial M} &= \frac{1}{n} \frac{d\mathbf{X}}{dt} \\ \frac{\partial M}{\partial a} &= -\frac{3}{2} \sqrt{\frac{\mu}{a^5}} t\end{aligned}$$

With all the partial derivatives found for each data point, the Jacobian matrix can be formed and the Gauss-Newton matrix can be computed. Finding the eigenvalues and eigenvectors of the Gauss-Newton matrix will show how observable each parameter is to the Doppler residual.

See Appendix A for the Python code of the above maths broken down into separate functions.

3 Method

With all over the math described in the previous section turned into python functions, the pipeline shown in Figure 1 can be used to find the Gauss-Newton Matrix from the initial TLE and following Doppler shift measurements.

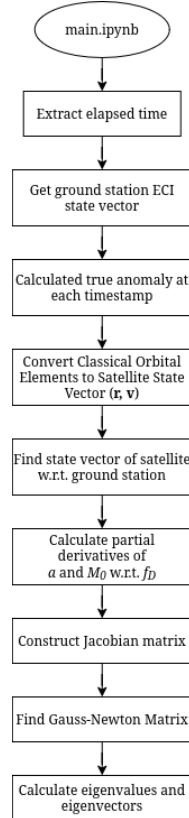


Figure 1: Block diagram of main code

3.1 Inputs

The two inputs required to produce Gauss-Newton matrix are an initial two line element set (TLE) and proceeding observations. For this example, the initial TLE is given as such:

```

1 60956U 98067WW 24256.99968611 .00235683 00000+0 34489-2 0 9990
2 60956 51.6339 240.0861 0011344 359.3789 15.7195 15.54180488 2268

```

From the TLE, the following orbital parameters and datetime stamp can be found:

Parameter	Value
Semi-major axis (a_{true})	6783182.731170822 m
Eccentricity (e_{true})	0.0011344
Inclination (i_{true})	51.6339 deg
RAAN (Ω_{true})	240.0861 deg
Arg of perigee (ω_{true})	359.3789 deg
Mean anomaly ($M_{0 \text{ true}}$)	15.7195 deg
ISO Datetime stamp	2024-09-12T23:59:32.880

Thus, the truth of the parameters in question is represented by:

$$\boldsymbol{\theta}_{\text{true}} = (M_{0 \text{ true}}, a_{\text{true}}) = (15.7195 \text{ deg}, 6783182.731170822 \text{ m})$$

The proceeding observations are imported from a `.xlsx` file, as seen in Appendix B. It provides the time of the observation (year, month, day, hour, minute, seconds as separate columns), the measured Doppler shift (Hz), and the central frequency (Hz).

The timestamp at each observation can be used with the initial orbital parameters to calculate it's respective true anomaly ν as well as the state vector of the groundstation ($\mathbf{X}_{gs} = [\mathbf{r}_{gs}, \mathbf{v}_{gs}]^T$). From there, the math previously shown can be used to get the Gauss-Newton matrix.

4 Results

4.1 Single Doppler Measurement

The single Doppler measurement used in this section is:

Field	Value
Year	2024
Month	9
Day	13
Hour	7
Minute	35
Second	5.8524243
Doppler Shift (Hz)	-9.73E+03
Centre Frequency (Hz)	4.38E+08

Field	Value
-------	-------

Using this as the input gave the following values

Ground Station State Vectors

$$\mathbf{r}_{gs} = \begin{bmatrix} -4022202.60251025 \\ -3632551.277273 \\ -3351356.54150797 \end{bmatrix} \text{ m} \quad \mathbf{v}_{gs} = \begin{bmatrix} 264.87946656 \\ -292.71712886 \\ -0.62329967 \end{bmatrix} \text{ m/s}$$

True Anomaly

$$\nu = 6.030164 \text{ rad}$$

Satellite State Vectors

$$\mathbf{r} = \begin{bmatrix} -3020280.32564041 \\ -6044413.75274605 \\ 500780.32767482 \end{bmatrix} \text{ m} \quad \mathbf{v} = \begin{bmatrix} 4470.73818599 \\ -1738.65978667 \\ 5990.43721675 \end{bmatrix} \text{ m/s}$$

Jacobian Matrix

$$J = \begin{bmatrix} 1.56007496 \times 10^1 & -4.20501806 \times 10^{-3} \end{bmatrix}$$

Gauss-Newton Matrix

$$G = \begin{bmatrix} 6.31288011 \times 10^2 & -2.00436196 \times 10^{-1} \\ -2.00436196 \times 10^{-1} & 6.36392078 \times 10^{-5} \end{bmatrix}$$

Eigenvalues

$$\lambda = \begin{bmatrix} 0.0 \\ 631.28807418 \end{bmatrix}$$

Eigenvectors

$$\mathbf{V} = \begin{bmatrix} -3.17503553 \times 10^{-4} & -9.99999950 \times 10^{-1} \\ -9.99999950 \times 10^{-1} & 3.17503553 \times 10^{-4} \end{bmatrix}$$

4.2 Multiple Doppler Measurements

All the Doppler measurements used in this section are shown in Appendix B.

If the values from the previous section are correct, then it can be assumed that the functions work as intended, and thus only the results from the Jacobian matrix onwards will be shown.

Jacobian Matrix

$$J = \begin{bmatrix} 1.56007496 \times 10^1 & -4.20501806 \times 10^{-3} \\ 1.55842813 \times 10^1 & -4.19973428 \times 10^{-3} \\ 1.54993371 \times 10^1 & -4.17257670 \times 10^{-3} \\ 1.53480081 \times 10^1 & -4.12458914 \times 10^{-3} \\ 1.52995799 \times 10^1 & -4.10933696 \times 10^{-3} \\ 1.51958858 \times 10^1 & -4.07684685 \times 10^{-3} \\ 1.51439129 \times 10^1 & -4.06064734 \times 10^{-3} \\ 1.51395790 \times 10^1 & -4.05929904 \times 10^{-3} \\ 1.51143425 \times 10^1 & -4.05145554 \times 10^{-3} \\ 1.51101183 \times 10^1 & -4.05014396 \times 10^{-3} \end{bmatrix}$$

Gauss-Newton Matrix

$$G = \begin{bmatrix} 2.34234008 \times 10^2 & -6.29233512 \times 10^{-2} \\ -6.29233512 \times 10^{-2} & 1.69034631 \times 10^{-5} \end{bmatrix}$$

Eigenvalues

$$\lambda = \begin{bmatrix} 7.59107722 \times 10^{-11} \\ 2.34234025 \times 10^2 \end{bmatrix}$$

Eigenvectors

$$\mathbf{V} = \begin{bmatrix} -2.68634557 \times 10^{-4} & -9.99999964 \times 10^{-1} \\ -9.99999964 \times 10^{-1} & 2.68634557 \times 10^{-4} \end{bmatrix}$$

4.3 Standardisation

Eigenvalues and eigenvectors will only be meaningful if the values of G are in the same units. At the moment, each value in G currently has different units, as a result of the nature of the Jacobian matrix:

$$G = J^T J = \begin{bmatrix} \frac{\text{Hz}}{\text{deg}} \\ \frac{\text{Hz}}{\text{m}} \end{bmatrix} \begin{bmatrix} \frac{\text{Hz}}{\text{deg}} & \frac{\text{Hz}}{\text{m}} \end{bmatrix} = \begin{bmatrix} \frac{\text{Hz}^2}{\text{deg}^2} & \frac{\text{Hz}^2}{\text{deg m}} \\ \frac{\text{Hz}^2}{\text{deg m}} & \frac{\text{Hz}^2}{\text{m}^2} \end{bmatrix}$$

Thus, G needs to be scaled before analysing the eigenvalues and eigenvectors so that the directions are in comparable units. This is done with a diagonal scaling matrix D :

$$D = \text{diag}(d_{M_0}, d_a) = \begin{bmatrix} d_{M_0} & 0 \\ 0 & d_a \end{bmatrix}$$

And the scaled Gauss-Newton matrix $\tilde{G}$ is:

$$\tilde{G} = D^T G D$$

By making $d_{M_0} = \frac{1}{\sqrt{G_{11}}}$ and $d_a = \frac{1}{\sqrt{G_{22}}}$, $\tilde{G}_{11}$ and $\tilde{G}_{22}$ will be 1, making the effect of M_0 and a in the same scale and now comparable to each other:

$$\tilde{G} = \begin{bmatrix} 1 & \frac{G_{12}}{\sqrt{G_{11}G_{22}}} \\ \frac{G_{21}}{\sqrt{G_{11}G_{22}}} & 1 \end{bmatrix}$$

The off-diagonal term, $\tilde{G}_{12} = \tilde{G}_{21}$, is the correlation between the sensitivities of M_0 and a .

New eigenvalues and eigenvectors can be calculated from the standardised Gauss-Newton matrix.

4.3.1 Single Doppler Measurement

Gauss-Newton Matrix

$$\tilde{G} = \begin{bmatrix} 1.0 & -1.0 \\ -1.0 & 1.0 \end{bmatrix}$$

Eigenvalues

$$\tilde{\lambda} = \begin{bmatrix} 0.0 \\ 2.0 \end{bmatrix}$$

Eigenvectors

$$\tilde{\mathbf{V}} = \begin{bmatrix} -0.70710678 & -0.70710678 \\ -0.70710678 & 0.70710678 \end{bmatrix}$$

4.3.2 Multiple Doppler Measurements

Gauss-Newton Matrix

$$\tilde{G} = \begin{bmatrix} 1.0 & -0.99913623 \\ -0.99913623 & 1.0 \end{bmatrix}$$

Eigenvalues

$$\tilde{\lambda} = \begin{bmatrix} 8.63771042 \times 10^{-4} \\ 1.99913623 \end{bmatrix}$$

Eigenvectors

$$\tilde{\mathbf{V}} = \begin{bmatrix} -0.70710678 & -0.70710678 \\ -0.70710678 & 0.70710678 \end{bmatrix}$$

5 Discussion

The Doppler model, $f_D(M_0, a)$, is a complicated, nonlinear relationship involving Kepler's equations, rotations and the ground station geometry. It has been linearised by replacing the model with the tangent (a first-order Taylor expansion) near a chosen point ($\boldsymbol{\theta}_{\text{true}} = (M_{0\text{ true}}, a_{\text{true}})$):

$$f_D(\boldsymbol{\theta}_{\text{true}} + \boldsymbol{\delta}) \approx f_D(\boldsymbol{\theta}_{\text{true}}) + J\boldsymbol{\delta}$$

As the linearisation is done at the point of truth, the squared error is zero, meaning $E \approx \frac{1}{2}\boldsymbol{\delta}^T G \boldsymbol{\delta}$, so G describes the shape of a quadratic bowl around the true parameters and is an approximation of the Fisher Information Matrix, which shows how observable each combination of parameters are from the data. The greater the change in the loss is, the more information that direction carries. The eigenvectors are the directions in the parameter space, where each column is a different combination of the parameters (M_0 and a). The eigenvalues are the steepness along each axis, where a larger eigenvalue of a direction shows that changing it greatly affects the predicted Doppler shift, and a near-zero eigenvalue means the predicted Doppler shift barely changes along that direction, so the data cannot distinguish parameter sets that differ only along it.

For the single measurement, the eigenvalues show us that there is one direction in the parameter space that greatly informs the Doppler shift prediction ($\lambda_2 = 2.0$), and one direction that does not ($\lambda_1 = 0.0$). This makes sense as one measurement cannot independently determine two parameters.

For a 2×2 standardised Gauss-Newton matrix of the form:

$$\tilde{G} = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$$

The eigenvalues will always be $\lambda = 1 \pm \rho$ and the eigenvectors will sit along one of the diagonals, $[1, 1]$ or $[1, -1]$ (or either of their negative counterparts). In this case, $\rho = -1$, resulting in $\lambda = (0, 2)$ The eigenvector associated with λ_1 and λ_2 respectively are:

$$\mathbf{V}_1 \approx \begin{bmatrix} -0.707 \\ -0.707 \end{bmatrix} \quad \mathbf{V}_2 \approx \begin{bmatrix} -0.707 \\ 0.707 \end{bmatrix}$$

Both eigenvectors give equal weighting to M_0 and a . $\mathbf{V}_1$, the unobservable direction, corresponds to both M_0 and a increasing or decreasing together, while the strongly observable eigenvector, $\mathbf{V}_2$, corresponds to one increasing while the other one decreases. So rather than the Doppler shift being sensitive to M_0 while giving no information on a , the single measurement constrains an equal combination of the two parameters, leaving the opposing direction completely free.

Using multiple observations, we see $\lambda_1 \approx 0.00086$ and $\lambda_2 \approx 1.999$. They have shifted a little bit but are still near their single-measurement value. Their respective eigenvectors are:

$$\mathbf{V}_1 \approx \begin{bmatrix} -0.707 \\ -0.707 \end{bmatrix} \quad \mathbf{V}_2 \approx \begin{bmatrix} -0.707 \\ 0.707 \end{bmatrix}$$

This matches that of the eigenvectors of the single measurement because of the properties of the standardised Gauss-Newton matrix. Despite there being more data, using

multiple observations in this case seems to not give much more information on the solution, as shown by the small shift in the eigenvalues. This is because most of the rows in J are approximately proportional to each other.

The multi-observation's struggle to observe more information is likely because the ten measurements span over 52s, under 1% of the roughly $\frac{2\pi}{n} \approx 5560$ s orbital period. The satellite wouldn't have moved very far within this time frame and thus all the information past the first row is mostly redundant and won't help form any new meaningful relationships.

5.1 Implications for Project

This exercise has shown that a single pass cannot resolve both a and M_0 . Even when multiple observations are taken over the pass, almost no new information is gained because every observation in the pass carries the same information. If a model were to be built around the current information as inputs, then the parameters the model outputs would be unreliable, as it is only able to constrain the solution in one direction.

As the model is further developed, observability checks on the parameters should be done before trusting a fit. The smallest eigenvalue should be calculated as a confidence check on how reliable the result is.

Further testing could be done in the following situations to see if the observability improves:

- How many passes / how much time is enough? (Could plot the lowest eigenvalue vs no. passes)
- Temporal Diversity - Multiple passes over the same ground station
 - Would spreading out the observations over several passes (hours or days apart) change the near-zero eigenvalue?
- Spatial Diversity - Single pass across multiple ground stations
 - Would two or more ground stations observing the same short time window add new independent information, or would it suffer the same problem as same-station single pass?
- Test across four situations:

	Single Pass	Multiple Passes
Single GS	SP-SG	MP-SG
Multi GS	SP-MG	MP-MG

A Python Code

```
1 import numpy as np
2 import pandas as pd
3 from scipy.spatial.transform import Rotation as R
4
5 from astropy.time import Time
6 from astropy.coordinates import EarthLocation, GCRS
7 import astropy.units as u
8
9 MU_EARTH = 3.986004418*(10**14)
10 SPEED_OF_LIGHT = 299792458
11
12 #-----
13 # Anomaly Propagation:
14 #   M0 -> M(t) -> E -> nu
15 #-----
16
17 # Calculate mean motion (n)
18 def nCalc(a, mu=MU_EARTH):
19     # Returns the mean motion in rads/s
20     return np.sqrt(mu / a**3)
21
22 # Solve Kepler's equation  $M = E - e \sin(E)$  for E, via Newton-
23   Raphson
24 def ECalc(M_deg, e, tolerance=1e-8):
25     # Inputs:
26     #   M - Mean anomaly (degrees), scalar or array-like
27     #   e - Eccentricity
28     #   tolerance - default to 1e-8
29     # Outputs:
30     #   E - Eccentric anomaly
31
32     M_deg_arr = np.atleast_1d(np.asarray(M_deg, dtype=float))
33     M_rad = np.deg2rad(M_deg_arr)
34
35     # Initial guess
36     E = np.where(((M_rad > -np.pi) & (M_rad < 0)) | (M_rad > np
37         .pi),
38                 M_rad - e, M_rad + e)
39
40     for i in range(len(E)):
41         E_i = E[i]
42         while True:
43             sin_E = np.sin(E_i)
44             cos_E = np.cos(E_i)
45             next_E = E_i + (M_rad[i] - E_i + e * sin_E) / (1 -
46                 e * cos_E)
47             abs_diff = abs(next_E - E_i)
48             E_i = next_E
49             if abs_diff < tolerance:
```

```

47         break
48     E[i] = E_i
49
50     return E[0] if E.size == 1 else E
51
52 # Propagate mean anomaly from a reference epoch to one or more
    observation
53 #     times (given as ISO 8601 strings), then solve for eccentric
    anomaly
54 def propagateM02E(elapsed_sec, M0_deg, n, e, tolerance=1e-8):
55     # Inputs:
56     #     elapsed_sec - Elapsed time since reference epoch t0 (s)
    , i.e. (t - t0).sec
57     #     M0_deg - initial mean anomaly (deg)
58     #     n - mean motion (rads/s)
59     #     e - eccentricity
60     # Outputs:
61     #     E - eccentric anomaly (rad)
62
63     n_deg_s = np.rad2deg(n)
64     M_deg = M0_deg + n_deg_s * elapsed_sec
65
66     return ECalc(M_deg, e, tolerance=tolerance)
67
68 # Calculate true anomaly (nu)
69 def nuCalc(E, e):
70     # Inputs:
71     #     E - Eccentric anomaly (radians)
72     #     e - Eccentricity
73     # Outputs:
74     #     nu - True Anomaly (radians)
75     nu = np.arctan2(np.sin(E) * np.sqrt(1-e**2), np.cos(E) - e)
76     return nu
77
78
79 #-----
80 # COE2RV:
81 #     Convert Classical Orbital Elements (COE) to
82 #     Cartesian Position and Velocity vectors (RV)
83 #-----
84 def COE2RV(coe, mu=MU_EARTH):
85     # INPUT:
86     #     coe is an array of the Keplerian Orbital Elements
87     #     a - semi-major axis (m)
88     #     e - eccentricity
89     #     i - inclination (deg)
90     #     node - right ascension of the ascending node (deg)
91     #     arg - argument of perigee (deg)
92     #     nu - true anomaly (radian)
93     #     mu - gravitational parameters (=GM). Default set to the
    value for Earth
94

```

```

95     # OUTPUT:
96     #   r - position vector of satellite
97     #   v - velocity vector of satellite
98
99
100    a, e, i, node, arg, nu = coe
101
102    sin_nu = np.sin(nu)
103    cos_nu = np.cos(nu)
104
105    # Calculate semiparameter (p)
106    p = a * (1-e**2)
107
108    # Perifocal Coordinate System
109    r_PQW = np.zeros(3)
110    v_PQW = np.zeros(3)
111
112    r_PQW[0] = (p * cos_nu) / (1 + e*cos_nu)
113    r_PQW[1] = (p * sin_nu) / (1 + e*cos_nu)
114
115    v_PQW[0] = -1 * np.sqrt(mu/p) * sin_nu
116    v_PQW[1] = np.sqrt(mu/p) * (e + cos_nu)
117
118    # Rotation Matrix
119    R_node_z = R.from_euler('z', np.deg2rad(node)).as_matrix()
120    R_i_x     = R.from_euler('x', np.deg2rad(i)).as_matrix()
121    R_arg_z   = R.from_euler('z', np.deg2rad(arg)).as_matrix()
122    R_total   = R_node_z @ R_i_x @ R_arg_z
123
124    # Rotate Perifocal to IJK
125    r_IJK = R_total @ r_PQW
126    v_IJK = R_total @ v_PQW
127
128    return r_IJK, v_IJK, R_total
129
130    #-----
131    # Time conversion
132    #-----
133    def ISOString(year, month, day, hour, minute, second):
134        # Inputs:
135        #   year, month, day, hour, minute - integer
136        #   second - float
137        # Output:
138        #   dt_str - ISO 8601 datetime string
139        dt_str = (
140            f"{int(year):04d}-{int(month):02d}-{int(day):02d}T"
141            f"{int(hour):02d}:{int(minute):02d}:{second:09.6f}"
142        )
143        return dt_str
144
145    # Get astropy.time.Time object of a specific row from a
146    DataFrame

```

```

146 # Apply to whole dataframe:
147 #     df["Time"] = df.apply(getTime, axis=1)
148 def getTime(row, t_scale='utc'):
149     # Input:  row of DataFrame that has the columns: "Year", "
150             Month", "Day", "Hour", "Minute", "Second"
151     #         t_scale - time scale, default 'utc'
152     # Output: astropy.time.Time object for the inputted row
153     iso_str = ISOString(
154         row["Year"], row["Month"], row["Day"],
155         row["Hour"], row["Minute"], row["Second"]
156     )
157     return Time(iso_str, scale=t_scale)
158
159 #-----
160 # Groundstation Vectors:
161 #   Geodetic coords -> ECI
162 #-----
163 def groundstationECI(lat_deg, lon_deg, alt_m, t):
164     # Inputs:
165     #   lat_deg - latitude of groundstation in degrees
166     #   lon_deg - longitude of groundstation in degrees
167     #   alt_m   - altitude of groundstation in metres
168     #   t       - astropy.time.Time object (single time, or
169                 array of times)
170     # Outputs:
171     #   r_gs - cartesian position vector(s) of groundstation (m
172             )
173     #   v_gs - cartesian velocity vector(s) of groundstation (m
174             /s)
175     station = EarthLocation(lat=lat_deg*u.deg, lon=lon_deg*u.
176                             deg, height=alt_m*u.m)
177
178     pos, vel = station.get_gcrs_posvel(obstime=t)
179
180     r_gs = pos.xyz.to(u.m).value
181     v_gs = vel.xyz.to(u.m/u.s).value
182     return r_gs, v_gs
183
184 #-----
185 # Doppler Shift Calculations from r and v
186 #-----
187 # rho - position vector of the satellite w.r.t groundstation
188 def rhoCalc(r, r_gs):
189     return r-r_gs
190
191 # rho_hat - unit vector of rho
192 def rho_hatCalc(rho):
193     return rho / np.sqrt(rho.dot(rho))
194
195 # v_rel - velocity vector of the satellite w.r.t groundstation

```

```

193 def v_relCalc(v, v_gs):
194     return v-v_gs
195
196 # frequency-to-range-rate scale factor
197 def kCalc(f_c, c=SPEED_OF_LIGHT):
198     return f_c/c
199
200
201 # Doppler shift calculation
202 # NOTE: NOT USED?
203 def fDCalc(k, rho_hat, v_rel):
204     return k * rho_hat * (-v_rel)
205
206 # Residual Calculation
207 # NOTE: NOT USED?
208 def residual(y_pred, y_true):
209     return y_pred-y_true
210
211 #-----
212 # Gradient Calculations
213 #-----
214 # Relationship between Doppler shift and satellite's cartesian
    state
215 def dfD_dXCalc(k, v_rel, rho, rho_hat):
216     mag_rho = np.sqrt(rho.dot(rho))
217
218     I3 = np.eye(3)
219
220     dfD_dr = -k * v_rel.T @ ( I3/mag_rho - (np.outer(rho, rho))
        /(mag_rho**3) )
221     dfD_dv = -k * rho_hat.T
222     return np.concatenate([dfD_dr, dfD_dv])
223
224
225 # Derivative of the satellite's cartesian state
226 def dX_dtCalc(v, r, mu=MU_EARTH):
227     mag_r = np.sqrt(r.dot(r))
228     dv_dt = -mu * r / (mag_r**3)
229
230     return np.concatenate([v, dv_dt])
231
232 # Initial Mean Anomaly Gradient
233 def dfD_dM0Calc(dfD_dX, dX_dt, n):
234     # Inputs:
235     #   dfD_dX - Change in Doppler measurement w.r.t.
        satellite's cartesian state (1x6 array)
236     #   dX_dt - Change in satellite's cartesian state w.r.t.
        time (6x1 array)
237     #   n - Mean motion (rad/s)
238     # Outputs:
239     #   dfD_dM0 - Change in Doppler shift w.r.t. Initial Mean
        Anomaly (Hz/deg)

```

```

240     dM_dt_reciprocal = 1 / np.rad2deg(n)
241
242
243     return dfD_dX @ dX_dt * dM_dt_reciprocal
244
245 # Semi-Major Axis Gradient
246 def dfD_daCalc(dfD_dX, dX_dt, elapsed_sec, rot_mat, n, a, e, E,
247               nu, mu=MU_EARTH):
248     # Inputs:
249     #   dfD_dX      - Change in Doppler measurement w.r.t.
250                     satellite's cartesian state (1x6 array)
251     #   dX_dt       - Change in satellite's cartesian state w.r.
252                     .t. time (6x1 array)
253     #   elapsed_sec - Elapsed time since reference epoch t0 (s)
254                     , i.e. (t - t0).sec
255     #   rot_mat     - Perifocal-to-ECI rotation matrix (from
256                     COE2RV)
257     #   n           - Mean motion (rad/s)
258     #   a           - Semi-major axis (m)
259     #   e           - Eccentricity
260     #   E           - Eccentric anomaly (rad)
261     #   nu          - True anomaly (rad)
262     #   mu          - Gravitational parameter (m3/s2) (
263                     Defaults to Earth's mu value)
264
265     # Outputs:
266     #   dfD_da      - Change in Doppler shift w.r.t semi-major axis
267     # NOTE: For calculations in this function, M is in radians
268
269     nu_arr = np.array([np.cos(nu), np.sin(nu), 0])
270     E_arr = np.array([-np.sin(E), np.sqrt(1-e**2) * np.cos(E),
271                      0])
272
273     dr_da_M = (1 - e*np.cos(E)) * rot_mat @ nu_arr
274     dv_da_M = -n / (2- 2*e*np.cos(E)) * rot_mat @ E_arr
275
276     dX_da_M = np.concatenate([dr_da_M, dv_da_M])
277     dX_dM = dX_dt / n
278     dM_da = -3/2 * np.sqrt(mu / (a**5)) * elapsed_sec
279
280     dX_da = dX_da_M + dX_dM * dM_da
281
282     return dfD_dX @ dX_da
283
284 #-----
285 # Jacobian Calculations
286 #-----
287 def Jacobian(dfD_dM0, dfD_da):
288     # Create Jacobian Matrix
289     # Inputs:
290     #   dfD_dM0 - Initial mean anomaly gradient (scalar or Nx1
291               array)
292     #   dfD_da  - Semi-major axis gradient (scalar or Nx1

```

```

    array)
284 # Outputs:
285 #   J - Outputs Nx2 array, where N is the number of
    observations
286
287 # Convert scalar values to 1D arrays
288 dfD_dM0 = np.atleast_1d(dfD_dM0)
289 dfD_da = np.atleast_1d(dfD_da)
290
291 N = len(dfD_dM0)          # Number of entries
292
293 # Create matrix
294 J = np.zeros((N, 2))
295 for i in range(N):
296     J[i] = np.array([dfD_dM0[i], dfD_da[i]])
297
298 return J
299
300 #-----
301 # Gauss-Newton Matrix
302 #-----
303 def GaussNewton(J):
304     # Create Gauss-Newton Matrix
305     # Inputs:
306     #   J - Nx2 Jacobian Matrix, where N is the number of
    observations
307     # Outputs:
308     #   G - 2x2 Gauss-Newton Matrix
309
310     N = J.shape[0]
311     return 1/N * J.T @ J
312
313 def standardisedGaussNewton_M0_a(J):
314     # Create Standardised Gauss-Newton Matrix
315     # Inputs:
316     #   J - Nx2 Jacobian Matrix, where N is the number of
    observations
317     # Outputs:
318     #   G_tilde - 2x2 Gauss-Newton Matrix
319
320     G = GaussNewton(J)
321     sigma_M0 = 1 / np.sqrt(G[0, 0])
322     sigma_a = 1 / np.sqrt(G[1, 1])
323     D = np.array([[sigma_M0, 0], [0, sigma_a]])
324
325     return D.T @ G @ D

```

B Fake Doppler Measurement Data

Year	Month	Day	Hour	Minute	Second	Doppler Shift (Hz)	Centre Frequency (Hz)
2024	9	13	7	35	5.8524243	-9.73E+03	4.38E+08
2024	9	13	7	35	7.619208978	-9.73E+03	4.38E+08
2024	9	13	7	35	16.70989313	-9.75E+03	4.38E+08
2024	9	13	7	35	32.81289115	-9.77E+03	4.38E+08
2024	9	13	7	35	37.94155477	-9.78E+03	4.38E+08
2024	9	13	7	35	48.88342118	-9.79E+03	4.38E+08
2024	9	13	7	35	54.34751622	-9.79E+03	4.38E+08
2024	9	13	7	35	54.80255137	-9.79E+03	4.38E+08
2024	9	13	7	35	57.45041013	-9.80E+03	4.38E+08
2024	9	13	7	35	57.89331211	-9.80E+03	4.38E+08

References

- Kelso, T.S. 1996. “Orbital Coordinate Systems, Part III.” CelesTrak, January. Accessed September 27, 2026. <https://celestrak.org/columns/v02n03/>.
- Martens, James. 2020. “New Insights and Perspectives on the Natural Gradient Method.” Pre-published, September 19, 2020. Accessed September 22, 2026. <https://doi.org/10.48550/arXiv.1412.1193>. <http://arxiv.org/abs/1412.1193>.
- Vallado, David A. 2022. *Fundamentals of Astrodynamics and Applications*. Fifth edition, first printing. Space Technology Library 21. Torrance, CA: Microcosm Press.